One logic, three controllers

Raspberry Pi 5 · ESP32 · Wiren Board 7.4

	Raspberry Pi 5 Model B	ESP32-DevKitC V4 (ESP32-WROOM-32)	Wiren Board 7.4
Controller	Raspberry Pi 5 — Linux computer	ESP32-DevKitC V4 — microcontroller, no OS	Wiren Board 7.4 — Linux automation controller (DIN rail)
Programming language	Python 3	MicroPython	JavaScript (wb-rules, ES5)
GPIO handling	gpiozero objects by BCM number	machine.Pin by GPIO number	no GPIO in the program: MQTT controls "device/control"
Sensor reading	library objects; analog through the MCP3008 (SPI)	internal ADC1 (read_uv) and dht.measure()	dev[] of Modbus and ADC controls; getError() for a lost sensor
Relay control	GPIO → relay module input	GPIO → relay module input	write a relay control → Modbus command to the WB-MR6C
Scheduling	while True + time.sleep(1); hour from time.localtime()	same loop; time.localtime()[3]	setInterval every second + setTimeout steps; new Date().getHours()
Communication	UART (CO_2), SPI (ADC), HTTP (Flask)	UART2 (CO_2), UART1 (Nextion)	RS-485 Modbus RTU + MQTT
Dashboard	terminal + Flask web page	serial monitor + Nextion screen	virtual device "farm" in MQTT + wb-rules log
Required modules	MCP3008 for analog, relay modules, 12 V supply	relay modules, separate 5 V supply, 12 V supply	WB-MR6C relays, Modbus air sensor, 12 V + 5 V supplies
Main program structure	settings → pins → functions → web → while True	settings → pins → functions → while True	settings → controls → virtual device → functions → rules → setInterval

The same step in the three programs (real lines)

Programming language

```
[Raspberry Pi 5 Model B]
from gpiozero import OutputDevice, Button, MCP3008

[ESP32-DevKitC V4 (ESP32-WROOM-32)]
from machine import Pin, ADC, UART

[Wiren Board 7.4]
defineRule("dosing_button", {
```

GPIO handling

```
[Raspberry Pi 5 Model B]
pump = OutputDevice(17)

[ESP32-DevKitC V4 (ESP32-WROOM-32)]
pump = Pin(26, Pin.OUT, value=0)

[Wiren Board 7.4]
var PUMP = "wb-mr6c_32/K1";
```

Sensor reading

```
[Raspberry Pi 5 Model B]
    ph = round(ph_adc.voltage * 3.5, 2)

[ESP32-DevKitC V4 (ESP32-WROOM-32)]
    ph = round(volts(ph_adc) * 3.5, 2)

[Wiren Board 7.4]
  ph = round2(dev[PH_VOLTS] * 3.5);
```

Relay control

```
[Raspberry Pi 5 Model B]
        pump.on()

[ESP32-DevKitC V4 (ESP32-WROOM-32)]
        pump.on()

[Wiren Board 7.4]
  if (dev[name] !== on) dev[name] = on;
```

Scheduling

```
[Raspberry Pi 5 Model B]
    hour = time.localtime().tm_hour

```

```
[ESP32-DevKitC V4 (ESP32-WROOM-32)]
    hour = time.localtime()[3]

[Wiren Board 7.4]
setInterval(control, 1000);
```

Communication

```
[Raspberry Pi 5 Model B]
co2_port = serial.Serial("/dev/serial0", 9600, timeout=1)

[ESP32-DevKitC V4 (ESP32-WROOM-32)]
co2_port = UART(2, baudrate=9600, tx=17, rx=16, timeout=300)

[Wiren Board 7.4]
var TEMP = "air-sensor_2/Temperature";
```

Dashboard

```
[Raspberry Pi 5 Model B]
app.add_url_rule("/status", "status", status)

[ESP32-DevKitC V4 (ESP32-WROOM-32)]
    screen.write(name + '.txt="' + str(text) + '"')

[Wiren Board 7.4]
defineVirtualDevice("farm", {
```

Main program structure

```
[Raspberry Pi 5 Model B]
while True:

[ESP32-DevKitC V4 (ESP32-WROOM-32)]
while True:

[Wiren Board 7.4]
function control() {
```